

CEDT-Assignment-style Reference

Every macro and environment this style provides, with a worked example
XXXXXXXXXXXX Your Name

Contents

1	Math shortcuts	1
2	Writing in Thai	1
3	Problems and sub-problems	2
4	Theorem-style environments	2
5	Solution, proof, and to-submit boxes	3
6	Code blocks	3
7	Figures	4
8	Tables	5
9	Multiple choice	5
10	Spacing helper	5

1 Math shortcuts

Common number sets and delimiters get short macros so you don't have to remember the amsmath spelling every time:

- Sets: $\mathbb{R}, \mathbb{C}, \mathbb{Z}, \mathbb{N}$ – `\R \C \Z \N`
- Delimiters: $|x|, \|v\|, (a+b), [a,b]$ – `\abs \norm \paren \sqbracket`
- From loaded packages: script $\mathscr{F}$ (`\mathscr`, `mathrsfs`), vectors $\overrightarrow{AB}$ (`\vv`, `esvect`), bras/kets $\langle\psi|\phi\rangle$ (`braket`), struck-through $\cancel{x}$ (`cancel`) – all scale with the 9pt body text.

For example: $\forall x \in \mathbb{R}, |x| \geq 0$, and $\|v\| = \sqrt{v_1^2 + v_2^2}$.

2 Writing in Thai

The document language is English, but Thai can be dropped in anywhere. This needs the XeLaTeX build (`.latexmkrc` already sets it up); it won't work under pdf/LuaTeX. Thai text must be *wrapped* – unwrapped Thai falls back to the Latin font and comes out as blank boxes.

`\textthai{...}` switches to Thai for a few words inline: อักษรไทยผสมกับภาษาอังกฤษ in the middle of an English sentence, or the other way round: ตัวแปร x มีค่าเป็นจำนวนเต็ม.

The `thai` environment is for whole paragraphs – it turns on the dictionary-based line breaker, so long Thai (which has no spaces between words) wraps correctly: ภาษาไทยไม่มีการเว้นวรรคระหว่างคำ การขึ้นบรรทัดใหม่จึงต้องอาศัยการตัดคำด้วยพจนานุกรม ซึ่ง XeTeX ทำให้โดยอัตโนมัติ ย่อหน้ายาวพอที่จะขึ้นบรรทัดใหม่หลายครั้ง และยังแทรกคณิตศาสตร์อย่าง $a^2 + b^2 = c^2$ กับตัวเลข 42 ได้ตามปกติ

It composes with everything else – a Thai problem statement, a Thai solution, Thai `\choices`:

Problem 1. จงพิสูจน์ว่าผลบวกของจำนวนเต็มคู่สองจำนวนเป็นจำนวนคู่

1.1 ระบุนิยามของจำนวนคู่

1.2 Give the definition in English.

Solution

ให้ $a = 2m$ และ $b = 2n$ โดยที่ $m, n \in \mathbb{Z}$ จะได้ $a + b = 2(m + n)$ ซึ่งเป็นจำนวนคู่

Problem 2. ข้อใดต่อไปนี้ถูกต้อง

2.1 A. ถูก

B. ผิด

C. ไม่แน่ใจ

D. ไม่มีข้อมูล

The Thai face is Norasi (it ships a Latin set and a real italic/bold, so a stray English word inside Thai still typesets). Override it for one document after loading the style package:

```
\newfontfamily\thaifont[Script=Thai]{TH Sarabun New}
```

3 Problems and sub-problems

Problem 3. problem auto-numbers itself. Sub-items nest under subproblems (decimal, e.g. 2.1) by default:

3.1 Decimal sub-items look like this.

3.2 A sub-item can go one level deeper with Tsubsubproblems:

T3.2.1 A "T"-prefixed third nesting level – adjust the fixed "T" in the style file if it doesn't fit your subject.

T3.2.2 Another one at the same depth.

Or letter-labelled with subproblems_alpha instead:

a) Like this.

b) And this.

Problem 10. Passing a number, e.g. `\begin{problem}[10]`, sets/reuses that exact problem number instead of auto-incrementing – handy when your numbering has to match a printed assignment sheet. Auto-numbering after this continues from whatever the highest-used number is, skipping any manually reused ones.

4 Theorem-style environments

Six environments, rendered as callouts with a coloured left spine (indigo for claims, teal for definitions, slate for remarks). `theorem/lemma/proposition/corollary` share one counter, `definition` numbers separately, and `remark` is unnumbered. Each takes an optional name, `\begin{theorem}[Euclid]`, and `\label/\ref` give the bare number, so write “Theorem `\ref{...}`”.

Definition 4.1. An integer n is *even* if $n = 2k$ for some $k \in \mathbb{Z}$.

Theorem 4.1 (Fundamental theorem of arithmetic). *Every integer greater than 1 has a unique prime factorization, up to the order of factors.*

Lemma 4.2. *If n^2 is even, then n is even.*

Proposition 4.3. *The sum of two even integers is even.*

Corollary 4.4. *If n is even, then n^2 is even.*

Remark. The heading colour and spine are the only custom part; numbering and the shared counter behave like amsthm. By Lemma 4.2, cross-references resolve as usual.

5 Solution, proof, and to-submit boxes

Problem 11. Worked-through answers that aren't the graded deliverable pair with solution.

Solution

The full derivation goes here.

A second paragraph indents correctly too – CEDT-Assignment-style.sty restores \parindent inside every breakable box below (tcolorbox's breakable library, needed so long boxes can split across a page, zeroes it internally otherwise).

Problem 12. A problem that must be proven pairs with proofbox.

Proof

The proof goes here.

A second paragraph, indented the same way. □

Problem 13. A problem whose final answer must be called out for grading pairs with tosubmit – its corner tab reads "TO SUBMIT | Solution" rather than just "TO SUBMIT".

TO SUBMIT | Solution

This is what a grader should look at directly.

A second paragraph here indents too.

All three corner-tab labels shown so far – "Solution", "Proof", "TO SUBMIT | Solution" – are just each box's default; pass title=... through the optional argument to override one for a single use, no .sty edit needed (same mechanism as codingbox's [language=C++] below):

FINAL ANSWER | Solution

A custom label via \begin{tosubmit}[title={FINAL ANSWER | Solution}].

6 Code blocks

codingbox defaults to Python, but any language listings ships a \lstdefinlanguage for works via its language option:

```
1 print("Python is the default language")
```

```

1  #include <iostream>
2
3  int main() {
4      // Compute the first 10 Fibonacci numbers
5      int a = 0, b = 1;
6      for (int i = 0; i < 10; ++i) {
7          std::cout << a << " ";
8          int next = a + b;
9          a = b;
10         b = next;
11     }
12     return 0;
13 }

```

```

1  #!/usr/bin/env bash
2  # Print the first 10 Fibonacci numbers
3  a=0
4  b=1
5  for _ in $(seq 1 10); do
6      echo -n "$a "
7      next=$((a + b))
8      a=$b
9      b=$next
10 done

```

7 Figures

`\fig[width]{path}{caption}{label}` centers an image in a numbered, captioned, labelled figure float (width defaults to $0.7\text{\linewidth}$):

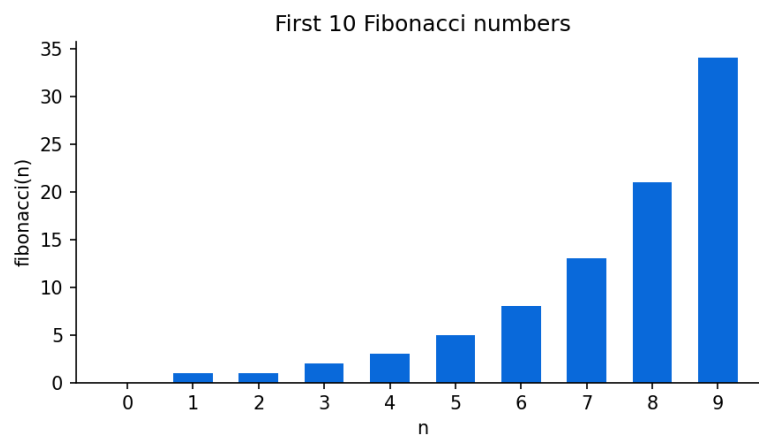
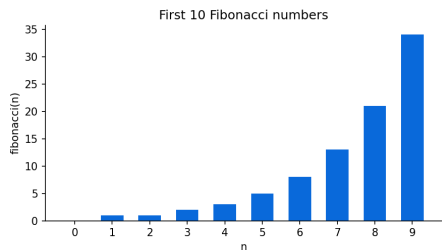


Figure 1: A numbered, captioned figure.

The starred form, `\fig*[width]{path}{}{}`, drops the caption/label entirely – just the centered image (the last two arguments are still required syntactically, but ignored, so pass `{}`):



8 Tables

`\begin{simpletable}<colspec><caption><label>` wraps a booktabs-style table with a numbered caption/label – write rows as normal and add `\midrule` after the header yourself:

<i>A</i>	<i>B</i>	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Table 1: Truth table for logical AND

9 Multiple choice

Problem 14. `\choices[<num choices>]{choice A, choice B, ...}` lays out options, auto-picking 4, 2, or 1 columns depending on how long the choices are (never more columns than there are choices):

14.1 Short choices sit four to a row.

- A. True
- B. False
- C. Maybe
- D. Unknown

14.2 Longer choices wrap down to two columns automatically.

- A. The probability increases
- B. The probability decreases
- C. The probability stays the same
- D. Not enough information to tell

14.3 Pass the choice count explicitly when it isn't 4.

- A. A
- B. B
- C. C
- D. D
- E. E
- F. F

10 Spacing helper

`\myspace[hor v][amount][unit]` wraps `\hspace/\vspace`; all three arguments are optional and default to `\myspace = \vspace{4mm}`. For example, `A\myspace[h][1][cm]B` gives A B (a 1cm gap between A and B on the same line).

APPENDIX 1

notebook-template | Code Solution

notebook-template

September 2, 2026

1 Example Notebook

This notebook is a worked example of `templates/nbconvert/cedt-assignment/` — the shared `jupyter nbconvert` LaTeX template in this repo. Convert it yourself with:

```
make nb-pdf NB=templates/notebook-template.ipynb
```

(from a subject repo that vendors this template, with `course-config.tex` filled in) to see the same header, 9pt font, and GitHub-style code boxes as `templates/assignment-template.tex`.

1.1 Problem

Write a function that returns the n -th Fibonacci number (0-indexed), then print the first ten values.

```
[1]: def fibonacci(n):  
    """Return the n-th Fibonacci number (0-indexed)."""  
    a, b = 0, 1  
    for _ in range(n):  
        a, b = b, a + b  
    return a  
  
# A short comment, to show off comment styling too  
values = [fibonacci(n) for n in range(10)]  
print("First 10 Fibonacci numbers:", values)
```

First 10 Fibonacci numbers: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

1.2 Notes

- Markdown cells render as normal LaTeX prose.
- Code cells get GitHub-style syntax highlighting via the `cedt-assignment` nbconvert template, matching `CEDT-Assignment-style.sty`'s `codingbox`.
- Both this source `.ipynb` and its compiled `.pdf` are committed here purely as a reference; nothing in this repo actually builds them itself (this repo isn't a subject repo — it has no `course-config.tex/vendor/` of its own).